

PORTADA

Análisis y Diseño de Sistemas II

**Semana 13 — Patrones GoF Estructurales II: Decorator, Proxy, Bridge, y
Laboratorio de implementación**

Universidad de Antioquia · Ingeniería de Sistemas



APERTURA

**Añadir una funcionalidad no debería obligarte
a crear una subclase nueva por cada combinación posible**

TRANSICIÓN

Cerrando la familia de estructurales GoF

El martes vieron **Adapter**, **Facade** y **Composite**, los tres primeros patrones estructurales del catálogo GoF — soluciones para componer clases y objetos en estructuras más grandes sin acoplar el diseño a implementaciones concretas. Hoy completan la familia con los tres patrones estructurales que faltan: **Decorator**, **Proxy** y **Bridge**.

Con estos seis (Adapter, Facade, Composite, Decorator, Proxy, Bridge) se cierra el catálogo completo de estructurales GoF — de los siete clásicos del libro original, esta semana cubre seis. La segunda mitad de la sesión es el **laboratorio de implementación**: cada equipo aplica al menos uno de los seis sobre su propio proyecto de Fábrica Escuela, con código real.

CONTRA EJEMPLO

El problema: una subclase por cada combinación de extras

```
class Cafe: ...  
class CafeConLeche(Cafe): ...  
class CafeConCanela(Cafe): ...  
class CafeConLecheYCanela(Cafe): ...  
class CafeConLecheYCanelaYChocolate(Cafe): ...  
# cada combinación nueva de extras exige una subclase nueva
```

Si cada "extra" (leche, canela, chocolate) se modela como una subclase distinta de `Cafe`, el número de clases crece más rápido que el número de extras: con solo tres extras ya hay ocho combinaciones posibles, cada una con su propia subclase. Además, la herencia fija la combinación **en tiempo de compilación** — no se puede decidir en tiempo de ejecución qué extras agregar a una taza concreta.

CONCEPTO

Decorator: añadir responsabilidades envolviendo, no heredando

Decorator (Erich Gamma et al., "Design Patterns", 1994) añade responsabilidades o comportamiento adicional a un objeto **individual**, de forma dinámica, envolviéndolo en uno o más objetos decoradores que implementan la **misma interfaz** que el objeto original.

Cada decorador guarda una referencia al objeto que envuelve (el original u otro decorador) y, al recibir una llamada, hace su parte adicional y **delega** el resto al objeto envuelto. Como el cliente solo conoce la interfaz común, puede tratar un objeto decorado exactamente igual que uno sin decorar.



CONCEPTO

La solución: decoradores apilables

```
class Cafe:                                # interfaz común
    def costo(self): return 3000

class ConLeche(Cafe):                      # implementa la misma interfaz
    def __init__(self, envuelto):
        self.envuelto = envuelto
    def costo(self):
        return self.envuelto.costo() + 800    # delega y suma

class ConCanela(Cafe):
    def __init__(self, envuelto):
        self.envuelto = envuelto
    def costo(self):
        return self.envuelto.costo() + 300

# uso: se apilan los decoradores en tiempo de ejecución
pedido = ConCanela(ConLeche(Cafe()))
pedido.costo()                            # 3000 + 800 + 300
```

CONCEPTO · ACLARACIÓN IMPORTANTE

Por qué esto evita la explosión de subclases

Con `ConLeche` y `ConCanela` como decoradores independientes, cualquier combinación se logra **componiendo** objetos en tiempo de ejecución (`ConCanela(ConLeche(Cafe()))`), sin declarar una clase nueva por combinación. Con N extras posibles, se necesitan solo N clases decoradoras, no una por cada subconjunto.

- El **orden** en que se apilan los decoradores puede importar (ej. aplicar un descuento antes o después de un impuesto da resultados distintos) — hay que decidirlo con intención, no dejarlo al azar.
- Cada decorador implementa la **misma interfaz** que `Cafe` : el cliente que recibe `pedido` no necesita saber cuántos decoradores hay ni en qué orden se aplicaron.

CONCEPTO · CUÁNDO USARLO

Cuándo vale la pena un Decorator — y en qué se diferencia de Composite

- Se necesita añadir comportamiento a un objeto **individual**, no a todos los objetos de la clase, y sin modificar su código fuente.
- Las combinaciones posibles de comportamiento adicional son **numerosas o cambian en tiempo de ejecución** — la herencia fija la combinación al escribir el código, Decorator la decide al ejecutarlo.

Diferencia con Composite (martes): Composite organiza objetos en una jerarquía **todo-parte** (una carpeta contiene archivos y otras carpetas) para tratar grupos e individuos de forma uniforme; Decorator envuelve **un solo objeto a la vez** para agregarle comportamiento, no para representar una jerarquía de contenido. Ambos usan composición y una interfaz común, pero resuelven problemas distintos.

CONCEPTO

Proxy: un sustituto que controla el acceso al objeto real

Proxy provee un objeto sustituto o representante que controla el acceso a otro objeto (el **objeto real**), interceptando las llamadas para agregar control adicional — sin que el cliente note la diferencia, porque el proxy implementa la **misma interfaz** que el objeto real.

A simple vista se parece a Decorator (ambos envuelven un objeto detrás de la misma interfaz), pero el propósito es distinto: Decorator **añade** comportamiento nuevo; Proxy **controla el acceso** al objeto que envuelve — puede incluso decidir no delegar la llamada en absoluto (ej. negarla por falta de permisos).



CONCEPTO

El proxy interceptando la llamada

```
class RepositorioDocumentos:      # interfaz común
    def obtener(self, id): ...

class RepositorioDocumentosReal(RepositorioDocumentos):
    def obtener(self, id):
        return cargarDesdeDisco(id)    # operación costosa

class RepositorioDocumentosProxy(RepositorioDocumentos):
    def __init__(self, real, usuario):
        self.real, self.usuario, self.cache = real, usuario, {}
    def obtener(self, id):
        if not self.usuario.tienePermiso(id):
            raise PermisoDenegado()      # control de acceso
        if id not in self.cache:
            self.cache[id] = self.real.obtener(id) # carga diferida
        return self.cache[id]

# el cliente usa el proxy igual que el objeto real:
repo = RepositorioDocumentosProxy(RepositorioDocumentosReal(), usuarioActual)
```

CONCEPTO · CUÁNDO USARLO

Subtipos conocidos de Proxy

El ejemplo anterior mezcla dos responsabilidades de proxy que la literatura suele nombrar por separado:

- **Proxy de protección:** valida permisos antes de permitir el acceso al objeto real (la validación de `tienePermiso` del ejemplo).
- **Proxy virtual:** retrasa la creación o carga de un objeto costoso hasta el momento en que realmente se necesita (**carga diferida** o *lazy loading*, el `cache` del ejemplo).

Ambos subtipos comparten la misma estructura — un objeto que implementa la interfaz del objeto real y decide cuándo y cómo delegarle la llamada — solo cambia la razón por la que interceptan. No es necesario memorizar más subtipos que estos dos para reconocer el patrón en código real.

CONCEPTO

Bridge: separar la abstracción de su implementación

Bridge separa una **abstracción** (qué hace algo) de su **implementación** (cómo lo hace), de modo que ambas puedan variar de forma independiente, evitando una jerarquía de herencia que combine dos dimensiones distintas de variación.

El problema que resuelve aparece cuando hay **dos ejes de variación** a la vez — ej. tipo de notificación (correo, SMS) × canal de envío (proveedor A, proveedor B) — y modelarlos con herencia obligaría a crear una subclase por cada combinación (`NotificacionCorreoProveedorA` , `NotificacionSMSProveedorB` , ...), la misma explosión combinatoria vista al inicio de la sesión, pero ahora entre dos jerarquías, no entre extras de un mismo objeto.



CONCEPTO

Dos jerarquías, unidas por una referencia

```
class CanalEnvio:                # interfaz de implementación
    def enviar(self, destino, texto): ...

class ProveedorA(CanalEnvio):
    def enviar(self, destino, texto): ...    # API del proveedor A

class ProveedorB(CanalEnvio):
    def enviar(self, destino, texto): ...    # API del proveedor B

class Notificacion:              # abstracción
    def __init__(self, canal: CanalEnvio):
        self.canal = canal          # el "puente" hacia la implementación
class NotificacionUrgente(Notificacion):
    def notificar(self, destino, mensaje):
        self.canal.enviar(destino, "URGENTE: " + mensaje)

# cualquier tipo de notificación combina con cualquier canal:
NotificacionUrgente(ProveedorA()).notificar("ana@x.com", "Falla")
```

TENSIÓN CONCEPTUAL

Bridge no es Adapter, aunque ambos "conectan" dos partes

Es común confundirlos porque ambos hacen que dos piezas de código colaboren a través de una interfaz intermedia. La diferencia está en **cuándo y por qué** aparece esa interfaz:

- **Adapter** (martes) se aplica **después**, sobre una interfaz **ya existente que no se puede cambiar** (una librería externa, un sistema legado): traduce esa interfaz a la que el cliente espera. Resuelve un problema de **compatibilidad**.
- **Bridge** se **diseña desde el principio**, antes de que exista el problema, para separar dos jerarquías que se sabe que van a variar de forma independiente. Resuelve un problema de **combinatoria de diseño**, no de compatibilidad con algo externo.



INTERACCIÓN

¿Cuál de los seis patrones estructurales aplica?

En el chat, para cada caso indiquen cuál de los seis patrones estructurales (Adapter, Facade, Composite, Decorator, Proxy o Bridge) es la mejor opción:

1. Necesitan agregar validación de permisos antes de que cualquier código acceda a un objeto de reporte financiero, sin que el resto del sistema note el cambio.
2. Un mismo cálculo de impuestos debe poder aplicarse solo, o combinado con un descuento, o con ambos, decidido en tiempo de ejecución.
3. Un tipo de figura (círculo, cuadrado) debe poder dibujarse con distintos motores de renderizado, y ambas listas — figuras y motores — van a seguir creciendo por separado.

5 minutos · respondan en el chat

SÍNTESIS

Los seis patrones estructurales GoF, completos

Patrón	Idea central
Adapter	Traducir una interfaz existente e incompatible a la que el cliente espera
Facade	Ofrecer una interfaz simple sobre un subsistema complejo
Composite	Tratar objetos individuales y grupos de objetos de forma uniforme, en una jerarquía todo-parte
Decorator	Añadir comportamiento a un objeto individual envolviéndolo, sin usar herencia por cada combinación
Proxy	Interponer un sustituto que controla el acceso al objeto real
Bridge	Separar una abstracción de su implementación para que ambas varíen independientemente

Con esto se completa el catálogo de estructurales GoF cubierto en el curso, iniciado el martes con Adapter, Facade y Composite.



TRANSICIÓN

Ahora, a implementarlos sobre el proyecto real

Los seis patrones se vieron con ejemplos de clase. El resto de la sesión es el **laboratorio de implementación de patrones estructurales**: cada equipo busca en su propio proyecto de Fábrica Escuela un punto donde la composición de clases podría beneficiarse de alguno de los seis patrones vistos (martes + hoy), y lo implementa con código real.



INSTRUCCIONES

Laboratorio: implementación de patrones estructurales

Trabajo en equipo — el mismo equipo del proyecto de Fábrica Escuela conformado desde la semana 2.

- Revisen el diseño actual de su proyecto y ubiquen **al menos un punto** donde la composición de clases sea rígida o incompatible: una interfaz externa que no encaja, un objeto que necesitaría comportamiento adicional según el caso, un acceso que debería controlarse, o dos dimensiones de variación mezcladas en una sola jerarquía.
- Elijan cuál de los **seis patrones estructurales** (Adapter, Facade, Composite, Decorator, Proxy o Bridge) resuelve mejor ese punto específico — no hace falta usar más de uno.



INSTRUCCIONES

Implementación y justificación

- Impleméntenlo con **código real** sobre su proyecto — o **pseudocódigo suficientemente detallado** si esa parte del sistema aún no está construida.
- Además del código, escriban una **nota corta** que justifique la elección: por qué ese patrón y no otro de los seis, y qué problema concreto de composición resolvía en su proyecto.
- Si consideraron un patrón y terminaron usando otro (ej. "pensamos en Decorator pero el problema real era el acceso controlado, así que usamos Proxy"), vale la pena dejarlo escrito — es evidencia de que la elección fue consciente.

Durante la sesión · trabajo en equipo sobre el proyecto propio

INSTRUCCIONES

Entregable y cierre del laboratorio

- **Formato exacto:** código funcional (o pseudocódigo detallado) que aplique al menos un patrón estructural al proyecto propio, más la nota corta de justificación, en el repositorio o herramienta de gestión del equipo.
- Debe quedar identificable qué **patrón** se aplicó, en qué clase u objeto, y **por qué** se eligió ese y no otro.
- Quien no alcance a terminar durante la sesión puede completarlo **antes de la próxima clase**.

Esta implementación es parte del insumo de diseño para el **Momento 2 — Diseño de software y propuesta de Arquitectura de Software (20%)**, que se evalúa la próxima semana. El miércoles de esta semana es Coworking (Planning Sprint 3), sin relación directa con este laboratorio.

SÍNTESIS

Qué debieron lograr en este laboratorio

1. Identificar un punto real de su proyecto donde la composición de clases podría mejorar.
2. Elegir, entre los seis patrones estructurales vistos (martes + hoy), el más adecuado para ese punto.
3. Implementarlo con código real o pseudocódigo suficientemente detallado.
4. Dejar por escrito por qué se eligió ese patrón y no otro de los seis.



CIERRE

Sigue la última familia GoF: los de comportamiento

Con esto se completa el catálogo de **patrones estructurales GoF** (Adapter, Facade, Composite, Decorator, Proxy, Bridge) aplicado al proyecto propio.

La **semana 14** abre la última familia de la Unidad 5: los **patrones GoF de Comportamiento** — el martes se ven Strategy, Observer y Command; el miércoles, State, Template Method e Iterator; y el viernes cierra con Antipatrones y el laboratorio de cierre de toda la Unidad 5.

